Parallel Savings Based Heuristics for the Delivery Problem

Kemal Altinkemer; Bezalel Gavish

Operations Research, Vol. 39, No. 3. (May - Jun., 1991), pp. 456-469.

Stable URL:
http://links.jstor.org/sici?sici=0030-364X%28199105%2F06%2939%3A3%3C456%3APSBHFT%3E2.0.CO%3B2-Z

Operations Research is currently published by INFORMS.

Your use of the JSTOR archive indicates your acceptance of JSTOR's Terms and Conditions of Use, available at http://www.jstor.org/about/terms.html. JSTOR's Terms and Conditions of Use provides, in part, that unless you have obtained prior permission, you may not download an entire issue of a journal or multiple copies of articles, and you may use content in the JSTOR archive only for your personal, non-commercial use.

Please contact the publisher regarding any further use of this work. Publisher contact information may be obtained at http://www.jstor.org/journals/informs.html.

Each copy of any part of a JSTOR transmission must contain the same copyright notice that appears on the screen or printed page of such transmission.

JSTOR is an independent not-for-profit organization dedicated to creating and preserving a digital archive of scholarly journals. For more information regarding JSTOR, please contact support@jstor.org.

http://www.jstor.org/
Thu Aug 24 16:48:11 2006

PARALLEL SAVINGS BASED HEURISTICS FOR THE DELIVERY PROBLEM

KEMAL ALTINKEMER

Purdue University, West Lafayette, Indiana

BEZALEL GAVISH

Vanderbilt University, Nashville, Tennessee

(Received December 1987; revisions received January, December 1989; accepted February 1990)

The delivery problem consists of finding a set of routes for a fleet of capacitated vehicles to satisfy the cargo delivery requirements of customers. The vehicles are located in a central depot, and have to fulfill the delivery requirements in a sequence that minimizes total delivery costs. Each vehicle tour starts and terminates at the central depot, and each node is supplied by exactly one vehicle. All vehicles have the same cargo carrying capacity. The paper presents parallel savings algorithms (PSAs) for generating feasible solutions to this problem. The new algorithms combine the savings approach, with matching based procedures. In computational tests the heuristic produces better solutions than the best known solutions for six problems out of a standard set of 14 difficult test problems. Augmented Lagrangian based lower bounding procedures are developed, and used to evaluate the quality of the solutions generated by PSAs. The lower bounds generated by the augmented Lagrangian are the tightest bounds known for delivery problems. The performance of the PSAs is also compared to tour partitioning based heuristics which have better worst case error bounds. The average quality of solutions generated by PSAs is shown to be significantly superior on large sets of test problems.

The delivery problem is one of a group of closely related problems called the vehicle routing problem (VRP). The delivery problem looks for the most economical way to fulfill the deterministic transportation requirements of customers by a fleet of vehicles. The requirements of customers could be pickups or deliveries. Each vehicle starts from a centralized depot, visits the set of customers assigned to it in an order that minimizes the trip cost and returns to the depot. Real-life applications of the delivery problem include: deliveries of products to supermarkets and department stores, picking up students by school buses, newspaper, laundry and mail distribution, doctors' housecall tours, preventive maintenance inspection tours, and the topological design of local access ring networks.

The delivery problem has been shown by Lenstra and Rinnooy Kan (1981) to be NP-hard, therefore, it is unlikely that there exists a polynomial-time algorithm for this problem. Computational experience with the delivery problem reveals that existing algorithms (Christofides, Mingozzi and Toth 1981) can solve problems with up to 25 nodes. Repeated attempts to solve larger problems to optimality in reasonable amounts of computational effort have failed. Due to the limited success of exact methods, much attention in the literature has been devoted to the development of heuristic algorithms.

In the next section, the problem is formulated and earlier research is summarized. In Section 2 a parallel savings algorithm (PSA) for the delivery problem is presented. The algorithm is based on the savings idea, which was first introduced by Clarke and Wright. The main change from the original savings approach is in replacing the sequential, single-tour merging procedure used in Clarke and Wright, by a matching-based procedure which merges multiple partial solutions in each step. This extension was first applied in Gavish and Altinkemer (1986) to capacitated minimal spanning tree problems. In Section 3, results of computational tests are reported with various parallel savings algorithms on a set of 14 test cases taken from the literature. These problems are known to be difficult to solve, and have been used extensively as a benchmark for comparing algorithms for solving the delivery problem. The PSA produces better solutions, than those previously known, to six of those problems. Section 4 presents two lower bounding procedures to the delivery problem. Computational tests reveal that these bounds are the best lower bounds known so far to this problem. Section 5 compares the empirical performance of the parallel savings heuristic to tour partitioning based heuristics, which have superior worst case error bounds. This comparison was done on a set of 125 randomly generated problems. The test data demonstrate that the average performance of tour partitioning based procedures is significantly inferior to parallel savings based heuristics.

Subject classifications: Integer programming: Lagrangian relaxation for delivery problems. Vehicle routing: heuristics for vehicle routing.

0030-364X/91/3903-0000$01.25
© 1991 Operations Research Society of America

1. THE DELIVERY PROBLEM

The delivery problem is to find a set of routes for a fleet of vehicles with known capacities to satisfy the deterministic cargo delivery requirements of customers. The vehicles are located in a central depot, and have to fulfill the delivery requirements in a sequence that minimizes the total delivery costs. Each vehicle tour starts and terminates at the central depot, and the load to be delivered to each node is supplied by exactly one vehicle. A set of nodes that is on the same tour (i.e., served by the same vehicle) form a cluster. The total delivery cost includes: operating expenses, fuel and labor. A mathematical formulation of the problem is presented below for the case in which the number of tours is not given in advance, and the cost matrix c_{ij} is symmetric. The following notation is used for formulating the problem:

n = the number of nodes in the problem;
V = the node set of the graph, $V = \{1, \ldots n,\}$;
$\tilde{V}$ = the node set excluding the center (node 1), $\tilde{V} = \{2, \ldots n,\}$;
E = the edge set of the graph;
Q = the vehicle capacity;
d_i = the load to be delivered to node i ($d_1 = 0$);
c_{ij} = the cost of travel from node i to node j, $c_{ii} = \infty$ for all $i \in V$.

These decision variables are used in the formulation:

$$x_{ij} = \begin{cases} 1 & \text{if arc } (i, j) \text{ is traversed by a vehicle} \\ 0 & \text{otherwise.} \end{cases}$$

Problem DP

Find variables x_{ij} that minimize

$$Z_{DP} = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} c_{ij} x_{ij} + \sum_{i=2}^{n} c_{i1} x_{i1}$$

subject to

$$x_{j1} + \sum_{i=1}^{j-1} x_{ij} + \sum_{i=j+1}^{n} x_{ji} = 2 \quad j = 2, \ldots, n-1 \tag{1}$$

$$x_{n1} + \sum_{j=1}^{n-1} x_{jn} = 2 \tag{2}$$

$$\sum_{j=2}^{n} x_{j1} \geqslant L_{\tilde{V}} \tag{3}$$

$$\sum_{i=1}^{n-1} \sum_{j=i+1}^{n} x_{ij} = n - 1 \tag{4}$$

$$\sum_{i \in S} \sum_{i<j \in S} x_{ij} \leqslant |S| - L_S \quad \text{for all } S \subseteq \{2, 3 \ldots n\} \tag{5}$$

$$x_{ij} = 0 \text{ or } 1 \quad \text{for all } i, j. \tag{6}$$

This formulation is a modification of the multiple traveling salesman problem in Gavish and Shrikanth (1987), and has been applied to the capacitated minimal spanning tree problem in Gavish (1985). The first and second sets of constraints ensure that exactly one vehicle enters and departs from every delivery node. The third constraint set states that an identical number of vehicles has to enter and leave the central depot. L_S is the optimal solution to the one-dimensional bin packing problem (Eilon and Christofides 1971), where the bin length is Q, and D_i, $i \in S$ are the lengths of items to be packed into the bins. The constraint in (3) provides a lower bound on the number of returning arcs to the depot and is redundant. However, it is helpful in the Lagrangian relaxation to tighten the lower bounds on the optimal solution value. Constraint (4) follows from the fact that any feasible solution must have n − 1 arcs in addition to the arcs directly returning to the depot. Constraint (5) ensures that no subtours can be formed without passing through the depot and the capacity constraint is not violated. There are other ways to represent the subtour elimination constraints, but constraint set (5) used in the formulation leads to the Lagrangian relaxation described in Section 4. The last set of constraints is the integrality constraints.

The reader is referred to Bodin et al. (1983) for a comprehensive survey of vehicle routing problems. The delivery problem solutions are classified into seven groups: The first group covers the cluster first–route second method to identify clusters of nodes assigned to the same vehicle, and a minimum cost tour for each cluster is computed. The second group is the route first–cluster second method that partitions an infeasible tour to find feasible routes. The third group is the savings or insertion method that adds a link according to some cost savings measure until all customers have been served. The fourth group describes the improvement or exchange method which perturbs the initial tours to find a better solution. The mathematical programming approach is the fifth group and is based on mathematical formulations of the problem. The sixth group covers the interactive optimization method which involves man-machine interaction, where an expert decides which direction the algorithm should take by changing some parameter values. The last group consists of exact algorithms which find the optimal solution. The next section outlines the set of algorithms against which the empirical performance of the parallel savings algorithm is compared.

The savings algorithm of Clarke and Wright (1964) belongs to the savings insertion group and has sequential and concurrent implementations. In the concurrent version, tours are formed simultaneously. Initially every

customer is served by a distinct vehicle. Satisfying the demands of customer i and j by the same vehicle will result in savings $s_{ij} = c_{1i} + c_{1j} - c_{ij}$. These savings are sorted in decreasing order. The algorithm merges customers i and j corresponding to the highest saving s_{ij} without violating the capacity restriction until no more mergers are possible. Although clusters grow simultaneously, only two clusters are merged at each step of the algorithm. The savings algorithm has a time complexity of $O(n^2 \log n)$ and a storage complexity of $O(n^2)$. In the sequential version, instead of calculating the entire savings matrix, an arbitrary node is picked as the initial cluster. At each step of the algorithm, the link corresponding to the highest savings from the last node added is selected, as long as this addition does not violate any constraints. Once the tour (cluster) cannot be expanded, a new tour is formed until all the nodes are served. For the sequential savings algorithm, Golden (1977) constructs an example where the ratio of the heuristic solution value to the optimal value is higher than $(6 \log n + 5)/21$.

Christofides, Mingozzi and Toth (1979) develop a two-phase algorithm. Phase one creates h routes according to a minimum cost insertion procedure. The second phase starts with h routes, and assigns customers to one of those routes by comparing the cost of inserting them to alternate routes. The algorithm terminates when every customer is served. Otherwise, the two phases are repeated until all customers are assigned to routes at the end of phase two. A second heuristic described in Christofides, Mingozzi and Toth (1979) is a tree search method. It is based on an exact branch-and-bound algorithm, and becomes a heuristic by using artificial fathoming values.

The generalized assignment heuristic (Fisher and Jaikumar 1981) is based on the idea that the mathematical formulation of the problem could be written as a generalized assignment problem. The objective function coefficients are the cost of the optimal traveling salesman tour over the set of nodes assigned to a vehicle. It is very complicated to write the cost coefficients beforehand, therefore, a linear approximation of the cost coefficients is used. First, a set of seed customers is chosen for each vehicle. Initially each vehicle serves only its assigned seed points. Following the formation of seed tours, the remaining customers are assigned to the seed tours by solving a generalized assignment problem.

A paper by Haimovich and Rinnooy Kan (1985) develops an iterated optimal tour partitioning algorithm (IOTP) for a special case of the delivery problem. All nodes have *identical* loads and therefore the vehicle capacity limitation is given by the maximum number of nodes a vehicle can serve. A feasible solution to the delivery problem is obtained by breaking an optimal traveling salesman tour into clusters that satisfy the vehicle capacity restriction. IOTP takes such a solution and by moving the endpoints of the subtours $1, 2, \ldots, n-1$ positions in the direction of a chosen orientation tries to improve the solution. IOTP has a worst case error bound of $2.5 - 1/Q$.

The parallel savings algorithm is presented in the next section. The algorithm assumes a symmetric cost matrix that satisfies the triangle inequality. These cost data are more general than that used in Haimovich and Rinnooy Kan. A major limitation of the new algorithm is the requirement to solve at each iteration, a traveling salesman problem. The TSP is formed over a subset of nodes whose total load does not exceed the vehicle's capacity. This is not a severe restriction because it is rare to find real-life applications of the delivery problem containing more than 20 stops per route. The reason for the small number of stops per route stems from the observations that a vehicle generally delivers multiple units per stop, time constraints exist for cargo loading and unloading, and the routes must be completed within a predetermined amount of time. TSPs consisting of 30–40 nodes and even problems involving hundreds (up to 500) of nodes can be solved in little computational effort as reported in Gavish and Shrikanth (1983). In the proposed algorithm, the first iterations contain a large number of clusters, but very few nodes in each cluster. As the algorithm progresses the clusters increase in size (the number of nodes), but the number of clusters goes down significantly.

Overall, the parallel savings algorithm (**PSA1**) is not polynomial. Thus, a second parallel savings algorithm (**PSA2**) that is fully polynomial is developed. It has a time complexity of $O(n^3)$ and uses an estimate of the savings realized by merging clusters in the matching procedure. Computational tests reveal that **PSA2** also generates solutions that are significantly better than previously published solutions.

2. PARALLEL SAVINGS HEURISTICS FOR THE DELIVERY PROBLEM

Parallel savings heuristics are matching based heuristics that belong to a class of greedy heuristics. They were first introduced and applied to capacitated tree systems by Gavish and Altinkemer. The only difference between the two classes of problems is that the underlying graphs in each cluster are spanning trees in capacitated tree systems, and tours in the delivery problem. Parallel savings heuristics merge multiple clusters of nodes at each iteration of the algorithm. The number of clusters merged at each iteration is determined by solving a

matching problem, maximizing the savings obtained in the present iteration. Concurrent versions of Clarke and Wright's savings algorithm, Golden (1977) and Ong and Moore (1984) also grow multiple clusters; however, at each step only two clusters are merged. In this section, a PSA for the delivery problem is presented.

The cost matrix of the problem is symmetric with $c_{ii} = \infty$ for all i; all arc costs are nonnegative and satisfy the triangle inequality; d_i is the load to be delivered to node i. The notation used is given by:

R_p = the set of nodes in cluster p; this is the set of nodes assigned to vehicle p;
$C(R_p)$ = the cost of the optimum traveling salesman tour over R_p and node 1 (the depot);
S_{pq} = the savings obtained by merging clusters p and q;
$\tilde{V}_i$ = the set of clusters at the end of iteration i not including the depot node (1);
V_i = the set of clusters at the end of iteration i including the depot node (1);
M_i = a set of dummy nodes added to the network at iteration i; $|M_i| = n - (i+1)T$ and $2 \leqslant T \leqslant 12$;
p_f = the first endpoint of the tour in cluster p;
p_l = the last end point of the tour in cluster p.

The PSA is applied to the original graph, with V as its node set and E as its edge set. In the first iteration each cluster consists of a single node of the original graph. When two clusters merge, the nodes in those clusters are served by a single vehicle. The optimal route serving the set of points in a cluster is obtained by solving a traveling salesman problem over the union of nodes in the cluster and the center. S_{pq}, the exact savings associated with merging clusters p and q, is given by $S_{pq} = C(R_p) + C(R_q) - C(R_p \cup R_q)$ when merging the two clusters does not violate the capacity restriction. Merging clusters p and q is nonadmissible, if combining the two clusters violates the capacity constraint. A maximum cost weighted matching problem is solved on the graph $G(V, E, S)$, where S is used as cost entries. New clusters are formed by merging the matched clusters. This iterative procedure is repeated until all remaining mergers are nonadmissible. A formal statement of the algorithm is presented below.

2.1. Parallel Savings Algorithm

Step 0. (Initialization) For $p = 1, \ldots, n$, set $R_p = \{p\}$, $i = 1$, $V_0 = V$, and choose a value for T.

Step 1. Form the dummy node set M_i, such that $|M_i| = n - (i+1)T$. For every p and q, $p < q$ do

$$S_{pq} = \begin{cases} 0 & p < q \text{ and } p, q \in M_i \\ -\infty & p < q \text{ and } p \in V_{i-1} \text{ and } q \in M_i. \end{cases}$$

For every p and q, $p < q$ and p, $q \in V_{i-1}$ do:

a. If $\sum_{i \in R_p \cup R_q} d_i > Q$ go to the next combination of p and q;

b. Otherwise, compute S_{pq} according to the formula

$$S_{pq} = \begin{cases} 0 & \text{if } p = 1, q = 1, \ldots, |V_{i-1}| \\ C(R_p) + C(R_q) - C(R_p \cup R_q) & \text{otherwise.} \end{cases}$$

Step 2. Solve the maximum weighted matching problem with $V_{i-1} \cup M_i$ as its node set, and S_{pq} as the cost entry for arc (p, q). Merge clusters p and q if arc (p, q) is in the matching cluster and p, $q \in V_{i-1}$ unless either p or q is node 1.

Step 3. If all mergers are nonadmissible STOP, otherwise increase i by one, update V_i to include all clusters and go to Step 1.

The savings computed in Step 1b are the exact savings when clusters p and q are merged.

In **PSA2** and **PSA3**, instead of solving a TSP to calculate the savings at each iteration, an estimate of the savings generated by combining two clusters is calculated. The savings formula used in **PSA2** and **PSA3** is:

$$S_{pq} = \max \begin{cases} c_{1p_f} + c_{1q_f} - c_{p_f q_f} \\ c_{1p_f} + c_{1q_l} - c_{p_F q_l} \\ c_{1p_l} + c_{1q_l} - c_{p_L q_l} \\ c_{1p_l} + c_{1q_f} - c_{p_L q_f} \end{cases}.$$

The formula underestimates the savings obtained by merging clusters p and q. The formula assumes that the tour in the merged cluster will be the combination of the tours in clusters p and q joined by the endpoints of the tours in those clusters. Under this assumption, the order of the nodes visited in the merged tour is not changed although rearranging them might improve the solution. The only reason for using an estimate instead of solving the traveling salesman problem is to obtain a polynomial algorithm.

PSA3 is an improvement of **PSA2**. In **PSA3**, after the final clusters are formed using **PSA2**, a TSP is solved for each final cluster formed. The tour generated by the optimal traveling salesman tour of that cluster is selected as the solution for that cluster.

When $T = 0$, depending on the node weights, the heuristic can terminate when the total load in each cluster is greater than $Q/2$, but far less than Q, resulting in underutilized vehicles. In this situation no more mergers are possible, vehicle utilization is low, and the solutions are far from optimality. In order to slow down the rate at which clusters are formed, a set of dummy nodes, M_i, is added to the network at each iteration. Every node in M_i is connected to every cluster by an arc with a cost of $-\infty$. These cost structures assure

that matchings to dummy nodes will have priority over matchings between cluster nodes, as a result $|M_i|$ matchings are formed between V_i and M_i, and $\lfloor |V_i| - |M_i|/2 \rfloor$ matchings between the real clusters. When $|M_i| = n - (i+1)T$, T matchings take place between the clusters in V_i. By varying the value of T we can control the rate in which clusters grow.

The following lemma shows that the savings S_{pq} are always nonnegative.

Lemma 1. *$S_{pq} \geqslant 0$ for all p, q.*

Proof. Assume that $R_p = \{j, \ldots k\}$, $R_q = \{l, \ldots m\}$ and $C(R_p) = c_{1j} + \cdots + c_{k1}$, $C(R_q) = c_{l1} + \cdots + c_{m1}$. If merging clusters p and q satisfies the capacity restriction, one way to form a route in the merged cluster is to take the two routes, eliminate links $(k, 1)$ and $(1, l)$ and add link (k, l) to the solution. Due to the triangle inequality, $c_{k1} + c_{1l} \geqslant c_{kl}$. The cost of this new solution cannot be higher than the sum of route costs in clusters p and q. Clearly

$$C(R_p \cup R_q) \leqslant c_{1j} + \cdots + c_{kl} + \cdots + c_{m1}$$
$$\leqslant C(R_p) + C(R_q)$$

because $C(R_p \cup R_q)$ corresponds to the cost of an optimal traveling salesman tour over the merged set. Hence, the savings are nonnegative.

Lemma 2 and Corollary 1 establish an upper bound on the number of iterations performed by **PSA1** when $T = 0$. Let R_p^m be the set of nodes in cluster p at the end of iteration m. $\tilde{V}_m$ is the set of clusters at the end of iteration m not including the center. $\tilde{V}_0$ is the set of clusters at the beginning of the algorithm, each cluster contains a single node and $V = \tilde{V}_0 + \{1\}$, where V is the set of nodes.

Lemma 2. *At the end of iteration m, $\sum_{p \in A_m^i} |R_p^m| > 2^{m+i}$ for all A_m^i when the capacity limit Q is given as the maximal number of nodes a route can have. A_m^i is any set of $2^i + 1$ distinct clusters at the end of iteration m for $i = 0, 1, \ldots \lfloor \log(\tilde{V}_m - 1) \rfloor$ such that $A_m^i \subset \tilde{V}_m$, $|A_m^i| = 2^i + 1$ and $|A_m^i| \leqslant |\tilde{V}_m|$.*

Proof. The maximal weighted matching problem with nonnegative cost entries generates a maximal cardinality matching as a solution. Due to Lemma 1, in each iteration any two unmatched vertices can be matched without decreasing the cost of the solution.

When $|\tilde{V}_0|$ is even, then all clusters at the end of the first iteration have exactly two nodes. In that case

$$\sum_{p \in A_1^i} |R_p^I| = 2^{I+i} + 2 > 2^{I+i}$$

for all A_1^i satisfying the conditions of the lemma. If $|\tilde{V}_0|$ is odd, one cluster consists of one node and all other clusters have exactly two nodes. For A_1^i, which consist of clusters having two nodes only, the above equality clearly holds. In other cases, some A_1^i contain the cluster having one node; for this case we have

$$\sum_{p \in A_1^i} |R_p^1| \geqslant 2^{I+i} + 1 > 2^{I+i}.$$

Hence, the lemma is correct for the first iteration. Now, assume that the lemma is also true at the end of iteration k, that is

$$\sum_{p \in A_k^i} |R_p^k| > 2^{k+i}$$

for all A_k^i satisfying the conditions of the lemma. Since from Lemma 1 the savings are nonnegative, depending on the number of clusters at the end of iteration k, there can be at most one unmatched cluster at iteration $k+1$. Hence, for any A_{k+1}^i at the end of iteration $k+1$, there is a corresponding set of clusters $\tilde{A}_k^{i+I}$ merged to form the clusters in A_{k+1}^i. For some cases, A_{k+1}^i might include a set of nodes in an additional cluster other than the set of nodes included in the set of clusters in $\tilde{A}_k^{i+I}$. In those cases, that additional cluster is excluded from $\tilde{A}_k^{i+I}$. As a result, we have

$$\sum_{p \in A_{k+1}^i} |R_p^{k+I}| \geqslant \sum_{q \in \tilde{A}_k^{i+I}} |R_q^k| > 2^{k+i+I}.$$

Hence, the lemma holds for iteration $k+1$.

Corollary 1. PSA1 *terminates after $\lceil \log Q \rceil$ iterations when the capacity restriction Q is given as the maximal number of nodes a tour can have.*

Proof. From Lemma 2 it follows that

$$|R_p^{\lfloor \log Q \rfloor}| + |R_q^{\lfloor \log Q \rfloor}| > 2^{\lfloor \log Q \rfloor}$$

for all pairs of p and q; $p, q \in \tilde{V}_{\lfloor \log Q \rfloor}$ at the end of iteration $\lfloor \log Q \rfloor$. If $\lfloor \log Q \rfloor = \lceil \log Q \rceil$, then additional mergers are not possible because they will violate the capacity restriction and the algorithm terminates. Again from Lemma 2 we have

$$|R_p^{\lfloor \log Q \rfloor}| + |R_q^{\lfloor \log Q \rfloor}| + |R_s^{\lfloor \log Q \rfloor}| > 2^{\lfloor \log Q \rfloor + 1}$$

for all triplets p, q and s; $p, q, s \in \tilde{V}_{\lfloor \log Q \rfloor}$ at the end of iteration $\lfloor \log Q \rfloor$. If $\lceil \log Q \rceil \neq \lceil \log Q \rceil$ all pairs of clusters that do not violate the capacity restriction will merge at iteration $\lceil \log Q \rceil$ except at most one cluster which will remain unmatched. Any two pairs of clusters formed at iteration $\lceil \log Q \rceil$ contain at least three clusters from iteration $\lfloor \log Q \rfloor$. From this inequality it follows that any more mergers will violate the capacity

restriction. Hence, the algorithm terminates after $\lceil \log Q \rceil$ iterations.

When the nodes have unequal weights (i.e., $\exists i, j \in \tilde{V}$ such that $d_i \neq d_j$) similar reasoning shows that the maximal number of iterations is bounded by $\lceil \log n \rceil$.

PSA1 finds the optimal solution for the delivery problem when the clusters are restricted to contain at most two nodes. The following lemma clarifies this special case.

Lemma 3. **PSA1** *finds the optimum for the delivery problem when the capacities are given with a maximum of two nodes per vehicle. If $H1(Q)$ is the* **PSA1** *solution with a capacity restriction of Q nodes per vehicle and $Z(Q)$ is the correspo nding optimal solution, then $H1(2) = Z(2)$.*

Proof. The arcs in the optimal solution could be divided into two groups; the first consists of the arcs connecting nodes to the central depot, and the second group is arcs that interconnect nodes to each other. The second group of arcs are exactly the arcs that correspond to the minimum cost matching on the original graph. Therefore, including those arcs achieves the maximum savings in the first iteration of the first PSA. (There is no ambiguity when n is even. If n is odd simply add the $(n+1)^{st}$ node such that $c_{1,n+1} = 0$ and $c_{i,n+1} = c_{i1}$ $i = 2, \ldots, n$).

2.2. A Numerical Example

In order to illustrate the parallel savings algorithm, we apply it to a problem consisting of ten locations; a unit load has to be delivered to each node. The depot is at location one and the vehicle capacity is four units. The distance matrix c_{ij} for the example is given in Table I.

Initially each cluster has one node (Figure 1) (*i.e.*, each node is served by a separate vehicle), so $R_p = \{p\}$ and $C(R_p) = 2c_{1p}$. Since each pair satisfies the capacity restriction, S_{pq} is calculated for each pair of clusters (notice that $S_{pq} = S_{qp}$). For the first iteration, the savings S_{pq} are given by

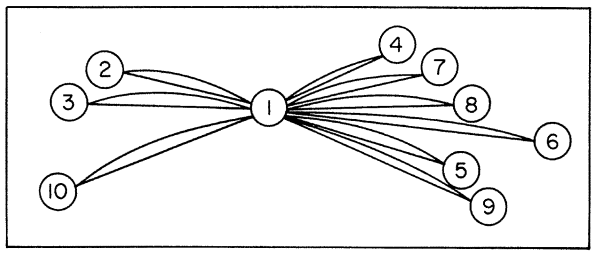

Figure 1. Example problem with 10 locations and the depot is at location 1; cost = 3,094.

$$C(R_p \cup R_q) = c_{pq} + c_{1p} + c_{1q}$$

$$S_{pq} = c_{1p} + c_{1q} - c_{pq}.$$

The maximum cost weighted matching problem with the cost matrix shown in Table II is solved. As a result, clusters that yield the maximum savings over the initial solution have been formed: $R_1 = \{1\}$, $R_2 = \{2\}$, $R_3 = \{3, 10\}$, $R_4 = \{4, 7\}$, $R_5 = \{5, 9\}$, $R_6 = \{6, 8\}$ (see Figure 2).

Since the total load in each cluster is at most two and merging them is still feasible, S_{pq} for each pair is calculated according to

$$S_{pq} = \begin{cases} 0 & \text{if } p = 1,\ q = 1, \ldots, |V_i| \\ C(R_p) + C(R_q) - C(R_p \cup R_q) & \text{otherwise.} \end{cases}$$

Table I
The Distance Matrix c_{ij}

	1								
2	164	2							
3	170	64	3						
4	104	261	274	4					
5	181	344	350	89	5				
6	243	400	413	140	74	6			
7	129	281	299	27	82	120	7		
8	169	323	339	65	59	79	41	8	
9	203	366	369	113	25	70	106	80	9
10	184	124	61	286	351	419	312	350	368

Table II
The Cost Matrix S_{pq} for the Weighted Matching Problem at Iteration 1

	1								
2	0	2							
3	0	270	3						
4	0	7	0	4					
5	0	1	1	196	5				
6	0	7	0	207	350	6			
7	0	12	0	206	228	252	7		
8	0	10	0	208	291	333	257	8	
9	0	1	4	194	359	376	226	292	9
10	0	224	293	2	14	8	1	3	19

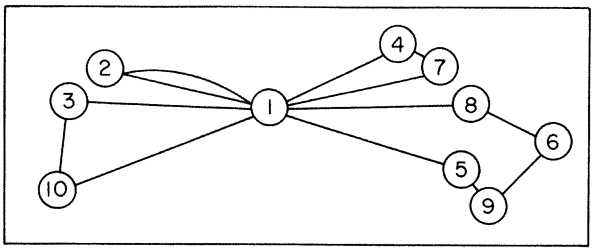

Figure 2. The components formed after the first iteration; cost = 1,903.

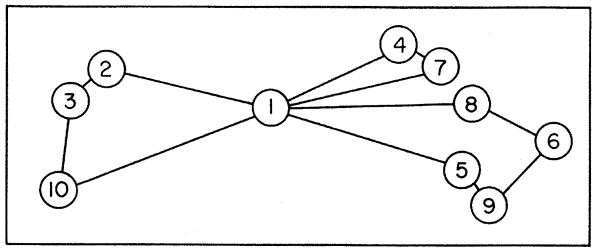

Figure 3. The components formed after the second iteration; cost = 1,257.

Thus, the savings S_{pq} are calculated by finding the optimal traveling salesman tour cost on $R_p \cup \{1\}$ and $R_q \cup \{1\}$. Next, the traveling salesman tour cost on the cluster $R_p \cup R_q$ and node 1 is found and subtracted from the sum of the first two. The exact savings are achieved by merging clusters p and q.

The maximum cost weighted matching problem is solved with the new cost matrix as shown in Table III, leading to the components $R_1 = \{1\}$, $R_2 = \{2, 3, 10\}$, $R_3 = \{4, 7\}$, $R_4 = \{5, 6, 8, 9\}$ (see Figure 3). This is the final iteration of the algorithm with a cost of 1,257. At this stage merging any two clusters violates the capacity constraint.

3. EMPIRICAL PERFORMANCE OF THE PARALLEL SAVINGS ALGORITHMS

The empirical performance of the parallel savings algorithms was evaluated on a set of 14 test problems taken from the literature. The problems range in size from 50–200 nodes. All the problems are planar and c_{ij}, the cost of traveling from node i to node j, is the Euclidean distance between points i and j, calculated according to the formula

$$c_{ij} = \lfloor \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2} + 0.5 \rfloor$$

Table III
The Cost Matrix S_{pq} for the Weighted Matching Problem at Iteration 2

	1				
2	0	2			
3	0	270	3		
4	0	12	2	4	
5	0	1	19	228	5
6	0	10	8	258	376

where (x_i, y_i) are the x and y coordinates of point i. The first ten problems are randomly generated, the last four are structured problems.

In all 14 problems, node loads are not equal. All vehicles have the same capacity, and are located at the center. There is no restriction on the number of vehicles used. The data for the first three problems are given in Christofides and Eilon (1969). The data for problem 4 is obtained by adding the customers of problems 1 and 3 with the center location and vehicle capacities of problem 3. Problem 5 is obtained by adding the first 49 customers of problem 2 to customers of problem 4 with the center location and vehicle capacities of problem 4. The first five problems do not have any time restrictions on route duration. Problems 6–10 are the same problems as the first five, with time constraints on route duration, and a ten unit drop-off time, which is incurred at each stop. The data for problems 11–14 are given in Christofides, Mingozzi and Toth (1979). All those problems are known to be difficult to solve, even obtaining good feasible solutions for them is a difficult task. The problem characteristics are given in Table IV.

The time constraints are managed by preventing the merger of two clusters when the sum of the dropoff time at each node in the emerging cluster and the travel time of the complete tour exceeds the route time restriction. **PSA1**was tested on the 14 test problems. Table V gives the cost of the solutions obtained by **PSA1** for different T values. For problem 2 a solution with the same solution value as the best known solution value has been obtained. For problems 5, 9, 10, 11 and 13 better solutions than the best previously known solutions have been generated. As can be seen from the table, no relationship was found between the quality of the solution and the value of T. Hence, each problem has to be solved with different T values, and the best solution generated is selected.

PSA1 requires solving a TSP over the nodes of every pair of clusters considered for merger, implying that

Table IV
Characteristics of 14 Test Problems

Problem No.	Problem Size	Truck Capacity	Time Limit on Tour	Drop-off Time	Problem Origin
1	51	160	∞	0	[1][a]
2	76	140	∞	0	[1]
3	101	200	∞	0	[1]
4	151	200	∞	0	[2][b]
5	200	200	∞	0	[2]
6	51	160	200	10	[1]
7	76	140	160	10	[1]
8	101	200	230	10	[1]
9	151	200	200	10	[2]
10	200	200	200	10	[2]
11	121	200	∞	0	[2]
12	101	200	∞	0	[2]
13	121	200	720	50	[2]
14	101	200	1040	90	[2]

[a][1] Christofides and Eilon (1969).
[b][2] Christofides, Mingozzi and Toth (1979).

PSA1 is a nonpolynomial algorithm. This is not such a severe restriction for problems with a relatively low a priori bound on the maximal number of stops in a tour. For problems involving more stops per tour, a fully polynomial algorithm **PSA2** can be used. **PSA2** generates solutions for problems 5, 10, 11 and 13 that are better than previous solutions found in the literature.

Solutions obtained by **PSA2** could be improved further by finding the optimal traveling salesman tour in each final cluster. **PSA3** is the same algorithm as **PSA2**, but at its completion a TSP is solved for each cluster. **PSA3** generates superior results than **PSA2**. Using **PSA3** better solutions were generated for problems 2, 5, 9, 10, 11 and 13 than the best known earlier solutions. **PSA2** has the worst solution quality among the three algorithms tested. Still, **PSA2** generates good clusters, as evidenced by the results of **PSA3**. When the best routes in each **PSA2** final cluster are computed, they are very close to the **PSA1** solutions, and for some cases **PSA3** leads to improved solutions. The best solutions for **PSA1**, **PSA2**, **PSA3**, and their corresponding T values are given in Table VI.

CPU times (in seconds) for **PSA1**, **PSA2** and **PSA3** on an IBM 3083 are given in Table VII. For all three algorithms, the cardinality of T changes from 2 to 13. For $T = 1$, **PSA2** is equivalent to Clarke and Wright's savings algorithm. We observe that when the cardinality of T is greater than 13, solutions generated by the PSA deteriorate as T increases. The CPU times given in Table VII consist of the sum of CPU times for 12 different T values for the same problem. Comparing CPU times for **PSA2** and **PSA3**, reveals that most of the computing time is spent on solving the matching problem. In this study, we were interested in investigating the quality of the solutions generated by the new procedures. No attempt was made to reduce solution times by paying

Table V
Results of the Computational Testing for
PSA1 With Different T Values

Problem		T									Best Solution
No.	Size	2	3	4	6	7	9	10	11	12	in Literature[a]
1	51	574	**560**	582	571	574	577	585	580	568	524 [2][e]
2[b]	76	**857**	866	894	889	883	975	875	895	870	857 [2]
3	101	870	903	892	881	**861**	874	892	872	877	833 [2]
4	151	1,112	1,117	1,127	1111	**1,085**	1,089	1,102	**1,085**	1,087	1,014 [2]
5[c]	200	1,357	1,357	1,364	**1351**	1,396	1,368	1,388	1,357	1,367	1,386 [1][d]
6	51	620	620	595	599	**583**	598	604	613	624	560 [2]
7	76	**943**	954	969	977	966	955	977	**1,016**	988	916 [2]
8	101	945	974	982	956	919	**913**	937	916	936	885 [2]
9[c]	151	1,270	1,320	1,274	1258	1,299	1,251	1,244	**1,211**	**1,260**	1,217 [1]
10[c]	200	1,571	1,573	1,534	1562	1,529	1,489	1,514	1,494	**1,464**	1,508 [1]
11[c]	121	1,052	**1,047**	1,089	1097	1,130	1,052	1,047	1,089	1,054	1,079 [1]
12	101	**834**	**834**	**834**	839	842	849	866	855	850	816 [2]
13[c]	121	1,559	**1,551**	1,560	1579	1,625	1,644	1,607	1,642	1,622	1,590 [1]
14	101	**874**	875	**874**	**874**	929	889	886	881	971	848 [1]

[a]Numbers in brackets represent the reference in which the solution was obtained.
[b]Same solution value as the best known solution value.
[c]Problems with better solutions than existing solutions.
[d][1] Christofides, Mingozzi and Toth (1979).
[e][2] Fisher and Jaikumar (1981).

Table VI
Best Solutions From **PSA1**, **PSA2** and **PSA3**

Problem		Best Solution Value and Corresponding T						Best Solution in Literature[a]
No.	Size	**PSA1**	T	**PSA2**	T	**PSA3**	T	
1	51	560	3	564	3	556	5	524 [2][d]
2[b]	76	857	2	862	4	**855**	4	857 [2]
3	101	861	9	868	7	860	7	833 [2]
4	151	1,085	7	1,104	4	1,089	7	1,014 [2]
5[b]	200	**1,351**	6	**1,370**	11	**1,353**	11	1,386 [1][c]
6	51	583	7	617	4	577	8	560 [2]
7	76	943	2	939	2	939	2	916 [2]
8	101	913	9	938	9	918	13	885 [2]
9[b]	151	**1,211**	11	1,236	10	**1,210**	10	1,217 [1]
10[b]	200	**1,464**	12	**1,499**	8	**1,469**	12	1,508 [1]
11[b]	121	**1,047**	3	**1,066**	2	**1,052**	2	1,079 [1]
12	101	834	2	845	5	834	2	816 [2]
13[b]	121	**1,551**	3	**1,572**	4	**1,552**	4	1,590 [1]
14	101	874	2	881	4	874	6	848 [1]

[a]Numbers in brackets represent the reference in which the solution was obtained.
[b]Problems with better solutions than existing solutions.
[c][1] Christofides, Mingozzi and Toth (1979).
[d][2] Fisher and Jaikumar (1981).

special attention to implementation details. It is possible to significantly reduce running times, by developing specialized matching codes, combining steps of the heuristic and using appropriate data structures.

4. LOWER BOUNDS ON THE DELIVERY PROBLEM

The numerical results reported in the previous section have improved results obtained in earlier studies. How far the new solution values are from the optimal solution can be determined by deriving either the optimal solutions or good lower bound values. Current methods for deriving lower bounds for the size of problems tested here provide poor lower bound values. In fact, this is one of the major drawbacks in the development of methods for solving delivery problems to optimality. We present two new lower bounding procedures for delivery problems. Computational tests reveal that they generate the tightest known lower bound values for these problems.

Table VII
CPU Times From **PSA1**, **PSA2** and **PSA3**

Problem		CPU Seconds (IBM3083)		
No.	Size	**PSA1**	**PSA2**	**PSA3**
1	51	70.13	21.40	22.45
2	76	171.56	65.96	67.13
3	101	590.81	169.31	171.52
4	151	2,162.72	589.02	591.08
5	200	3,071.49	1,150.37	1,153.17
6	51	71.89	21.47	22.52
7	76	173.46	66.84	67.92
8	101	598.17	171.02	173.20
9	151	2,173.08	594.64	596.76
10	200	3,087.73	1,156.14	1,158.82
11	121	1,237.70	224.38	226.79
12	101	592.28	168.43	170.61
13	121	1,241.06	226.03	228.17
14	101	597.93	170.31	172.57

To begin, a lower bound on the number of vehicles needed for a feasible solution is established. Let K_{DP} be the minimum number of vehicles needed for a delivery problem without any time restriction on tour length. K_{DP} satisfies the relationship

$$[K_{DP}-1]Q < \sum_{i=1}^{n} d_i \leqslant K_{DP}Q.$$

One way to derive a lower bound on the value of the optimal solution for the delivery problem is to solve a multiple traveling salesman problem, in which the number of salesmen is greater than or equal to K_{DP}. The cost of this solution is denoted by C_{DP}^{TSP}; C_{DP}^{TSP} is a lower bound since C_{DP}^{TSP} is the delivery problem with the capacity restriction relaxed.

A second method for generating lower bounds to the delivery problem is obtained by developing a Lagrangian relaxation of the formulation given in Section 1. In order to develop a Lagrangian relaxation of DP, the procedure developed in Gavish (1985) for computing lower

bounds to capacitated minimal spanning tree problems is followed. Constraint set (5) is divided into two subsets:

$$\sum_{i \in S} \sum_{i<j \in S} x_{ij} \leqslant |S| - 1$$

$$\text{for all } S \subseteq \{2, 3, \ldots n\} \; L_S = 1 \; |S| \geqslant 2 \qquad (5a)$$

$$\sum_{i \in S} \sum_{i<j \in S} x_{ij} \leqslant |S| - L_S$$

$$\text{for all } S \subseteq \{2, 3, \ldots n\} \; L_S \geqslant 2 \; |S| \geqslant 2. \qquad (5b)$$

The Lagrangian problem $L(\alpha, \pi)$ is generated by multiplying constraints (1), (2) and (5b) with Lagrange multipliers $\alpha = \{\alpha_2, \ldots \alpha_n\}$ and $\pi = \{\pi_1, \pi_2, \ldots\}$, such that α_i is unrestricted in sign and $\pi_i \geqslant 0$ for all i. Similar approaches have been applied to the multiple traveling salesman problem in Gavish and Shrikanth (1987), and to the capacitated spanning tree problems Gavish (1985). This leads to problem $L(\alpha, \pi)$ given below.

Problem L(α, π)

$$\begin{aligned} L(\alpha, \pi) = \min & \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} c_{ij} x_{ij} \\ & + \sum_{i=2}^{n} c_{i1} x_{i1} + \sum_{S \in \Psi} (|S| - L_S) \pi_S \\ & - \sum_{S \in \Psi} \sum_{i \in S} \sum_{i<j \in S} \pi_S x_{ij} \\ & + \sum_{j=2}^{n-1} \alpha_j \left(2 - x_{j1} - \sum_{i=1}^{j-1} x_{ij} - \sum_{i=j+1}^{n} x_{ji} \right) \\ & + \alpha_n \left(2 - x_{n1} - \sum_{j=1}^{n-1} x_{jn} \right) \end{aligned}$$

subject to (3), (4), (5a) and (6), where Ψ represents the set of unordered subsets of $\{2, \ldots, n\}$ with a cardinality greater than 1, such that each subset requires more than one vehicle (bin). For a given set of multipliers π and α, $L(\alpha, \pi)$ is a degree constrained minimal spanning tree problem (DCMST) with a cost matrix

$$\tilde{c}_{ij} = \begin{cases} c_{ij} - \sum_{k \in \Psi_{ij}} \pi_k - \alpha_j & \text{if } i = 1, \; j = 2, \ldots, n \\ c_{ij} - \sum_{k \in \Psi_{ij}} \pi_k - \alpha_i - \alpha_j & \text{if } i = 2, \ldots n-1 \\ & \quad j = i+1, \ldots, n \end{cases}$$

where Ψ_{ij} is the index set of subsets of Ψ that contain both nodes i and j. K_{DP} is a lower bound for $L_{\tilde{V}}$, and the degree of the depot has to be greater or equal to K_{DP}.

From earlier research on Lagrangian relaxation (Geoffrion 1974) it is known that

$$L(\alpha, \pi) \leqslant Z_{DP} \quad \text{for all } \alpha \text{ and } \pi \leqslant 0$$

$$L(\alpha^*, \pi^*) \leqslant Z_{DP}$$

where α^* and π^* are the multiplier values corresponding to the best Lagrangian value. Due to earlier positive experience with the subgradient optimization procedure (Held, Wolfe and Crowder 1974) it was used to compute improved multipliers and lower bounds. These lower bounds are directly applicable to the pure delivery problem. For problems with a time constraint imposed on route duration, a lower bound K^*_{time} on the minimum number of vehicles needed for a feasible solution, is given by

$$K^*_{time} = \max \left\{ K_{DP}, \left\lceil \frac{L(\alpha^*, \pi^*) + \sum_{i=2}^{n} t_i}{T} \right\rceil \right\}$$

where T is the time restriction, t_i is the drop-off or pick up time at node i, and $L(\alpha^*, \pi^*)$ is the best value obtained for the Lagrangian problem $L(\alpha, \pi)$, in which the cost matrix is identical to internode travel times.

Solutions for C^{TSP}_{DP} and $L(\alpha^*, \pi^*)$ can be used as lower bounds for the delivery problem. Optimal solutions to C^{TSP}_{DP}, the multiple salesman based bound, are obtained by the algorithm developed in Gavish and Shrikanth (1987). For the second method, the augmented Lagrangian algorithm developed in Gavish (1985) has been modified to solve $L(\alpha, \pi)$. The computational results of those experiments are presented in Table VIII. In most cases, $L(\alpha^*, \pi^*)$ provides tighter bounds than C^{TSP}_{DP}. It is interesting to note that in problems 12 and 14, C^{TSP}_{DP} based bounds were superior to those generated by $L(\alpha^*, \pi^*)$.

The lower bounds presented in Table VIII are the best ones known so far for these problems. The gap remaining between the best upper and lower bounds is still significant. Note that for problem 13 a large gap (117.8%) exists between the best feasible solution and the best lower bound, pointing out that these 14 problems are very difficult to solve.

5. COMPARING PSA HEURISTIC TO TOUR PARTITIONING BASED HEURISTICS

Altinkemer and Gavish (1987) developed a heuristic with a constant worst case error bound of $2 - 1/Q$ for delivery problems, where each vehicle could serve at most Q equal weight nodes. That algorithm is based on the *optimal* partitioning of an optimal traveling salesman tour; it will be referred to as OPTSP. OPTSP is similar

Table VIII
Lower Bounds to the 14 Test Problems

Problem		Lower Bound (LB)				
No.	Size	K_{DP}	C_{DP}^{TSP}	DCMST	Best Solution (Z_f) in Literature[a]	$\frac{Z_f - LB}{LB} \times 100$
1	51	5	464	500	524 [2]	4.8
2	76	10	635	742	855 **[PSA3]**	15.2
3	101	8	656	768	833 [2]	8.5
4	151	12	821	908	1014 [2]	11.7
5	200	17	964	1,101	1,351 **[PSA1]**	22.7
6	51	6	480	506	560 [2]	10.7
7	76	12	675	753	916 [2]	21.6
8	101	9	726	765	885 [2]	15.7
9	151	15	875	925	1,210 **[PSA1]**	30.8
10	200	19	1,003	1,114	1,464 **[PSA1]**	31.4
11	121	7	586	680	1.047 **[PSA1]**	53.9
12	101	10	745	734	816 [2]	11.2
13	121	10	586	712	1,551 **[PSA1]**	117.8
14	101	10	745	734	848 [1][b]	15.5

[a]Numbers in brackets represent the reference in which the solution was obtained.
[b][1] Christofides, Mingozzi and Toth (1979).
[c][2] Fisher and Jaikumar (1981).

to the iterated tour partitioning algorithm (IOTP) mentioned in Haimovich and Rinnooy Kan which has a worst case error bound of $2.5 - 1/Q$ when applied to the optimal traveling salesman tour. If optimal partitioning is applied to a traveling salesman tour obtained by Christofides' algorithm (Christofides 1976) the bound is $2.5 - 3/2Q$ for the Altinkemer and Gavish (1987b) heuristic. IOTP has a worst case error bound of $3 - 3/2Q$ when applied to Christofides' tour. Since OPTSP generates optimal partitions of a traveling salesman tour it always dominates solutions generated by IOTP.

In terms of worst case behavior, the following example shows that **PSA1**, **PSA2** and **PSA3** can generate a solution with an error ratio of 3. Suppose that the nodes are equidistant from the depot with a distance M. The vehicle capacity is $Q = 7$ and the total number of nodes is 28 (excluding the depot). They are clustered in 4 groups of 7 nodes each. The distance between the nodes within a cluster is ϵ and the distance between nodes in different clusters is 2M. **PSA1** could generate a solution shown in Figure 4 with a cost of $H(7) = 20M + 18\epsilon$, where each cluster contains four nodes and any other merging will violate the capacity restriction. The optimal solution to this problem is shown in Figure 5 and has a cost of $Z(7) = 8M + 24\epsilon$. The generalized case of the same example has a vehicle capacity of $Q = 2^i - 1$ and consists of 2^{i-1} clusters of $2^i - 1$ unit weight nodes for $i = 3, 4, \ldots$. The distance between the nodes within a cluster is ϵ, whereas the distance between nodes in different clusters is 2M. For the generalized case, the ratio of the **PSA1** solution to the cost of the optimal solution is given by

$$\frac{H(2^i - 1)}{Z(2^i - 1)} \leqslant \lim_{i \to \infty} \lim_{\epsilon \to 0} \frac{H(2^i - 1)}{Z(2^i - 1)}$$
$$= \lim_{i \to \infty} \frac{2(2^i - 1) + 2(2^{i-1} - 1)}{2^i} = 3.$$

This example implies that in terms of worst case performance, **PSA1** is inferior to tour partitioning based heuristics. A worst case behavior does not necessarily imply an inferior average behavior. In order to examine its empirical performance, **PSA1** has been tested against both OPTSP and optimal partitioning of Christofides' tour (OPC). One hundred and twenty five problems were created where customer locations have been generated

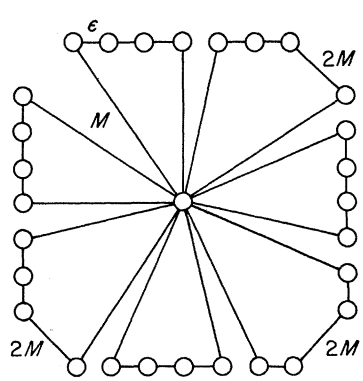

Figure 4. The **PSA1** solution; cost H(7) = 20M + 18ϵ.

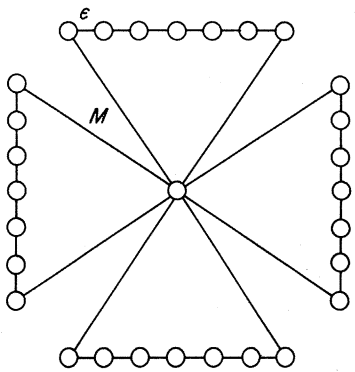

Figure 5. The optimal solution; cost Z(7) = 8M + 24ε.

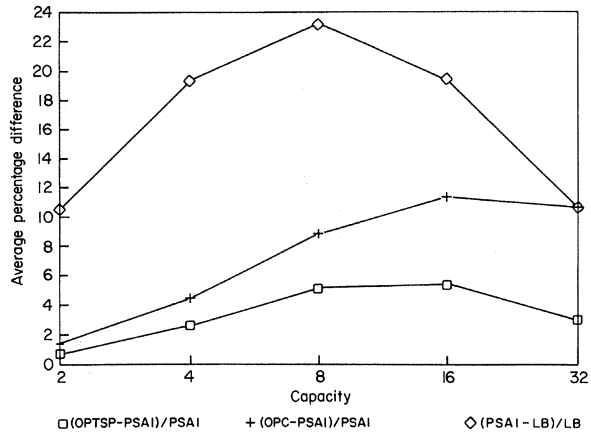

Figure 6. Average percentage difference between OPTSP, OPC, LB and **PSA1**.

randomly on a rectangle of 2,000 by 5,000, all with equal unit weights. The depot is located at the intersection of the diagonals. Each problem consists of 100 customers. The vehicle capacity restriction has been varied from 2 to 32 and for each case 25 problems have been generated. **PSA1** always produces better solutions than OPC and it outperforms OPTSP in 123 out of 125 cases. Both of the cases where OPTSP gave better results were cases with a high vehicle capacity ($Q = 32$). In the experiments, the variance of the percent difference increases when the capacity restriction increases. The average percent difference was the highest and lowest when the vehicle capacity was 16 and 2, respectively. **PSA1** finds the optimal solution when the capacity restriction is 2. The low percent difference of solution values from **PSA1** and the optimal partitioning algorithms means that for this data generation method, the arcs in the optimal traveling salesman tour are likely to be in an optimal solution for the delivery problem. The results of the experiments appear in Table IX. Figure 6 shows the average percent differences between OPTSP, OPC, LB and **PSA1**.

Lower bounds to these problems have been computed by the augmented Lagrangian method described in the previous section. The average percent difference between **PSA1** solution values and the lower bound values generated by the Lagrangian is 10.5% when the capacity restriction is 2. The difference increases as the capacity restriction increases and reaches a peak at a capacity restriction of 8, where it is equal to 23.1%. Beyond this value it decreases as the capacity restriction increases. The gap between a feasible solution value and the lower bound is still high, which means that either better lower bounding procedures have to be found or the heuristics can be improved. The gap of 10.5% between **PSA1** and the Lagrangian based bounds for a capacity restriction of 2, indicates that a major portion of this gap is due to the poor quality of the lower bounding procedure. To verify this conjecture, better lower bounding procedures will have to be developed.

Table X compares the OPC solutions with the best solutions for the 14 test problems used in Section 3. As expected, solutions from OPC are much worse than the

Table IX
Percent Differences Between OPTSP, OPC, LB and **PSA1**

		Q				
		2	4	8	16	32
Percent difference between	Max.	1.3	3.8	6.6	8.7	9.7
optimal partitioning of	Mean	0.7	2.6	5.1	5.4	3.0
optimal tour and **PSA1**	Min.	0.3	0.8	3.4	3.1	−12.2
Percent difference between	Max.	1.9	5.7	11.3	15.9	14.8
optimal partitioning of	Mean	1.4	4.4	8.8	11.4	10.5
Christofides' tour and **PSA1**	Min.	0.9	2.9	6.7	8.4	4.6
Percent difference between	Max.	11.7	22.3	24.7	21.7	11.9
PSA1 and the lower bound	Mean	10.5	19.3	23.1	19.4	10.6
from the Lagrangian	Min.	8.4	16.8	21.5	17.0	8.1

Table X
Comparison of Best Solutions to OPC Solutions

Problem No.	Problem Size	Best Solution (Z_f) in Literature[a]	OPC Solution	Best PSA Solution
1	51	524 [2][c]	620	556
2	76	855 **[PSA3]**	941	855
3	101	833 [2]	964	860
4	151	1,014 [2]	1,240	1,085
5	200	1,351 **[PSA1]**	1,509	1,351
6	51	560 [2]	677	577
7	76	916 [2]	1,063	939
8	101	885 [2]	1,008	913
9	151	1,210 **[PSA3]**	1,399	1,210
10	200	1,464 **[PSA1]**	1,684	1,464
11	121	1,047 **[PSA1]**	1,132	1,047
12	101	816 [2]	975	834
13	121	1,551 **[PSA1]**	1,789	1,551
14	101	848 [1][b]	976	874

[a]Numbers in brackets represent the reference in which the solution was obtained.
[b][1] Christofides, Mingozzi and Toth (1979).
[c][2] Fisher and Jaikumar (1981).

solutions generated by parallel savings procedures. The major drawback of OPC is that the traveling salesman tour does not consider node weights when the large traveling salesman tour is constructed. Partitioning a traveling salesman tour to generate a feasible solution to unequal weight delivery problems is unlikely to produce good solutions.

Although optimal partitioning of the traveling salesman tour does not generate good solutions, a constant worst case error bound of $3 - 2/Q$ is established in Altinkemer and Gavish (1987a) for optimal partitioning applied to the unequal weight delivery problem.

6. CONCLUSION

In this paper, parallel savings based algorithms have been applied to the delivery problem. The empirical behavior of different variants of the parallel savings heuristic was tested on 14 test problems taken from the literature and better solutions than existing solutions have been found for six of those problems. It is difficult to generate good feasible solutions for these test problems. These problems are used extensively in the literature to compare the effectiveness of algorithms suggested for the delivery problem.

We could not establish a worst case error bound for PSA, but PSA solutions are compared against optimal tour partitioning algorithms which have a constant worst case error bound. On average, PSA performs much better than optimal tour partitioning algorithms.

The ideas presented in this paper have been used in Gavish and Shlifer (1979) in a failing attempt to develop a branch-and-bound based algorithm for solving the problem to optimality. The heuristics developed here can be extended to multiple depot problems, to multiple capacity and multicompartment vehicles (Gavish and Graves 1982). In Gavish and Altinkemer (1986), parallel savings algorithms are applied to topological design problems of local access tree networks (Gavish 1982). Optimal partitioning could be applied to capacitated tree problems in which the worst case error bound becomes $4 - 4/Q$ (Altinkemer and Gavish 1988). They could be also used for the topological design of ring networks, which are similar in structure to the design of a delivery system. It seems that parallel savings heuristics are applicable to many problems to which sequential greedy heuristics h ave been applied in the past.

ACKNOWLEDGMENT

Partial support for this research was furnished to the second author from the Dean's Fund for Faculty Research at the Owen Graduate School of Management of Vanderbilt University.

REFERENCES

ALTINKEMER K., AND B. GAVISH. 1987a. Heuristics for Unequal Weight Delivery Problems With a Fixed Error Guarantee. *Opns. Res. Letts.* **6**, 149–158.

ALTINKEMER K., AND B. GAVISH. 1987b. Heuristics for Equal Weight Delivery Problems With Constant Error Guarantees. *Trans. Sci.* (to appear).

ALTINKEMER K., AND B. GAVISH. 1988. Heuristics With Constant Error Guarantees for the Design of Tree Networks. *Mgmt. Sci.* **34**, 331–341.

BODIN L., B. GOLDEN, A. ASSAD AND M. BALL. 1983. Routing and Scheduling of Vehicles and Crews: The State of the Art. *Comput. Opns.* Res. **10**.

CLARKE, G., AND J. WRIGHT. 1964. Scheduling of Vehicles From a Central Depot to a Number of Delivery Points. *Opns. Res.* **12**, 568–581.

CHRISTOFIDES, N. 1976. Worst-Case Analysis of a New Heuristic for the Traveling Salesman Problem. Report 388, Graduate School of Industrial Administration, Carnegie Mellon University, Pittsburgh.

CHRISTOFIDES N., AND S. EILON. 1969. An Algorithm for the Vehicle Dispatching Problem. *Opns. Res. Quart.* **20**, 309–318.

CHRISTOFIDES N., A. MINGOZZI AND P. TOTH. 1979. The Vehicle Routing Problem. In *Combinatorial Optimization*. John Wiley & Sons, New York.

CHRISTOFIDES N., A. MINGOZZI AND P. TOTH. 1981. Exact

Algorithms for the Vehicle Routing Problem, Based on Spanning Tree and Shortest Path Relaxations. *Math. Prog.* **20**, 255–282.

ELION, S., AND N. CHRISTOFIDES. 1971. The Loading Problem. *Mgmt. Sci.* **17**, 259–268.

FISHER, M. L., AND R. JAIKUMAR. 1981. A Generalized Assignment Heuristic for Vehicle Routing. *Networks* **11**, 109–124.

GAVISH, B. 1982. Topological Design of Centralized Computer Networks-Formulations and Algorithms. *Networks* **12**, 355–377.

GAVISH, B. 1985. Augmented Lagrangian Based Algorithms for Centralized Network Design. *IEEE Trans. Commun.* **33**, 1247–1257.

GAVISH, B., AND K. ALTINKEMER. 1986. A Parallel Savings Heuristic for the Topological Design of Local Access Tree Networks. In *Proceedings of IEEE INFOCOM '86*, 130–139.

GAVISH, B., AND S. GRAVES. 1982. Scheduling and Routing in Transportation and Distribution Systems: Formulations and New Relaxations. Working Paper, Owen Graduate School of Management, Vanderbilt University, Nashville, Tenn.

GAVISH, B., AND E. SHILFER. 1979. An Approach for Solving a Class of Transportation Scheduling Problems. *Eur. J. Opns. Res.* **5**, 122–134.

GAVISH, B., AND K. SHRIKANTH. 1983. Efficient Branch and Bound Code for Solving Large Scale Traveling Salesman Problems to Optimality. Working Paper, Owen Graduate School of Management, Vanderbilt University, Nashville, Tenn.

GAVISH, B., AND K. SHRIKANTH. 1987. Optimal Solution Methods for Large Scale Multiple Traveling Salesman Problems. *Opns. Res.* **34**, 698–717.

GEOFFRION, A. M. 1974. Lagrangian Relaxation and Its Uses in Integer Programming. *Math. Prog. Study* **2**, 82–114.

GOLDEN, B. 1977. Evaluating a Sequential Vehicle Routing Algorithm. *AIIE Trans.* **9**, 204–208.

HAIMOVICH, M., AND A. RINNOOY KAN. 1985. Bounds and Heuristics for Capacitated Routing Problems. *Math. Opns. Res.* **10**, 527–542.

HELD, M., P. WOLFE AND H. P. CROWDER. 1974. Validation of Subgradient Optimization. *Math. Prog.* **6**, 62–88.

LENSTRA, J., AND A. RINNOOY KAN. 1981. Complexity of Vehicle Routing and Scheduling Problems. *Networks* **11**, 221–227.

ONG, H. L., AND J. B. MOORE. 1984. Worst-Case Analysis of Two Traveling Salesman Heuristics. *Opns. Res. Letts.* **2**, 273–277.